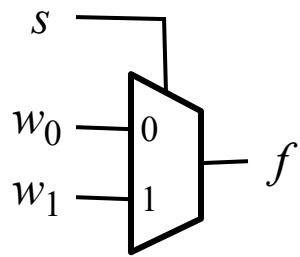


Digital Electronics
III Semester
B. Tech. (CSE)
Amrita School of Engineering, Bengaluru

Multiplexers

Multiplexers

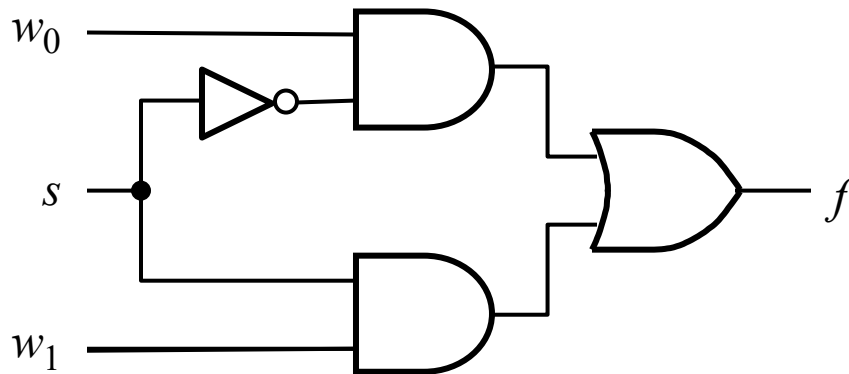
- A multiplexer has
 - N control inputs
 - 2^n data inputs
 - 1 output
- A multiplexer routes (or connects) the selected data input to the output.
 - The value of the control inputs determines the data input that is selected.



(a) Graphical symbol

s	f
0	w_0
1	w_1

(b) Truth table-compact



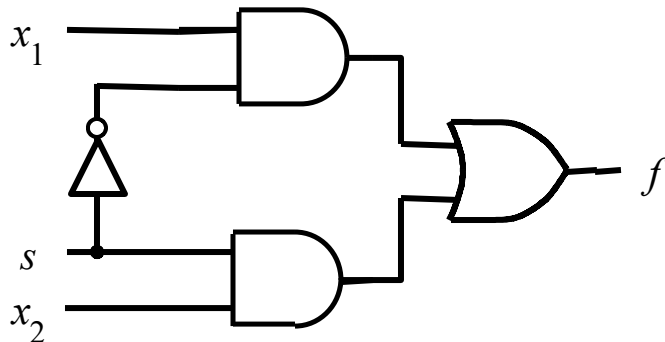
(c) Sum-of-products circuit

A 2-to-1 multiplexer.

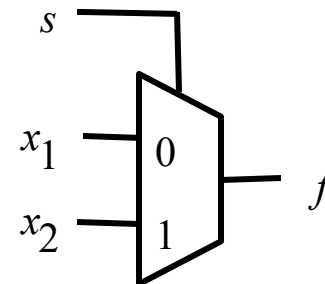
Truth Table for a 2-1 Multiplexer

s	x_1	x_2	$f(s, x_1, x_2)$
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

$$f(s, x_1, x_2) = \overline{s} x_1 + s x_2$$



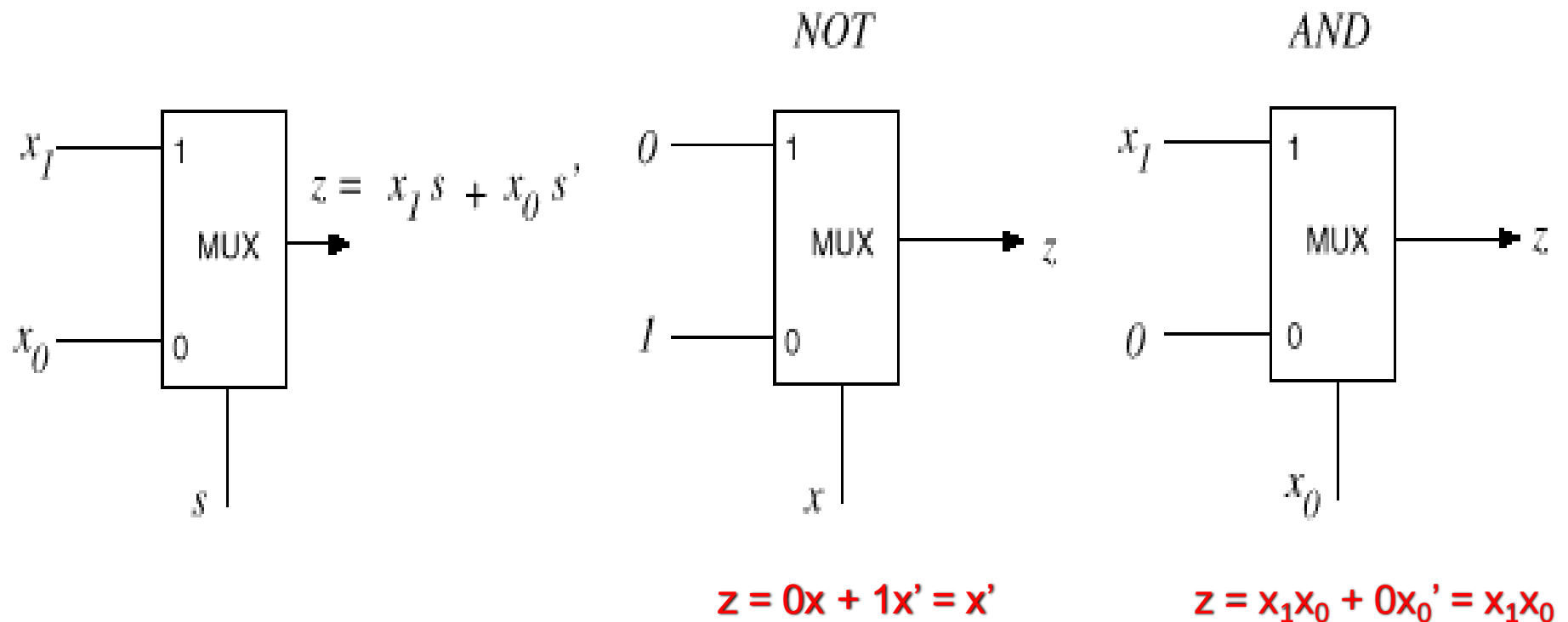
(b) Circuit



(c) Graphical symbol

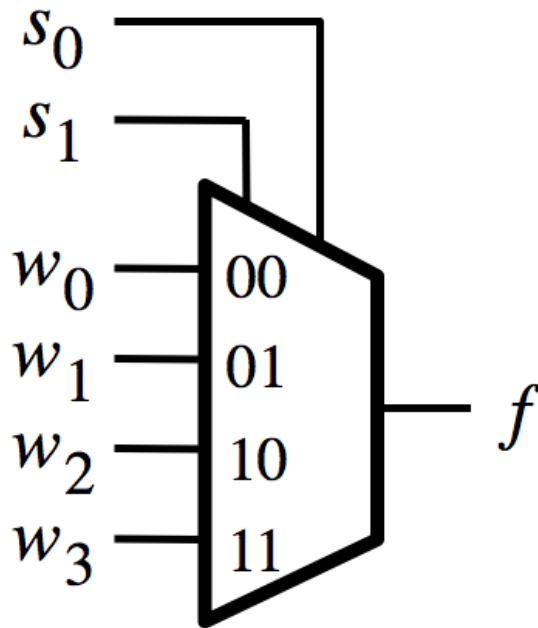
MUX as a Universal Gate

- We can construct AND and NOT gates using 2-to-1 MUXs. Thus, 2-to-1 MUX is a universal gate.
- Try OR Gate and NAND Gate



Graphical Symbol and Truth Table

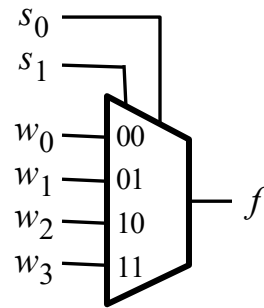
4 to 1 Multiplexer



(a) Graphic symbol

s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

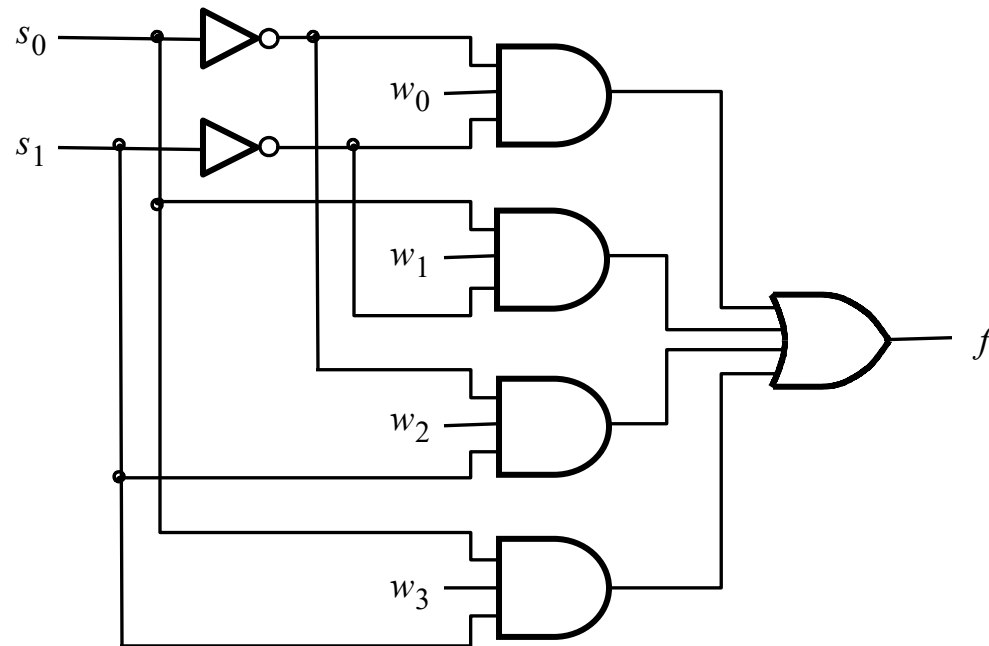
(b) Truth table



(a) Graphic symbol

s_1	s_0	f
0	0	w_0
0	1	w_1
1	0	w_2
1	1	w_3

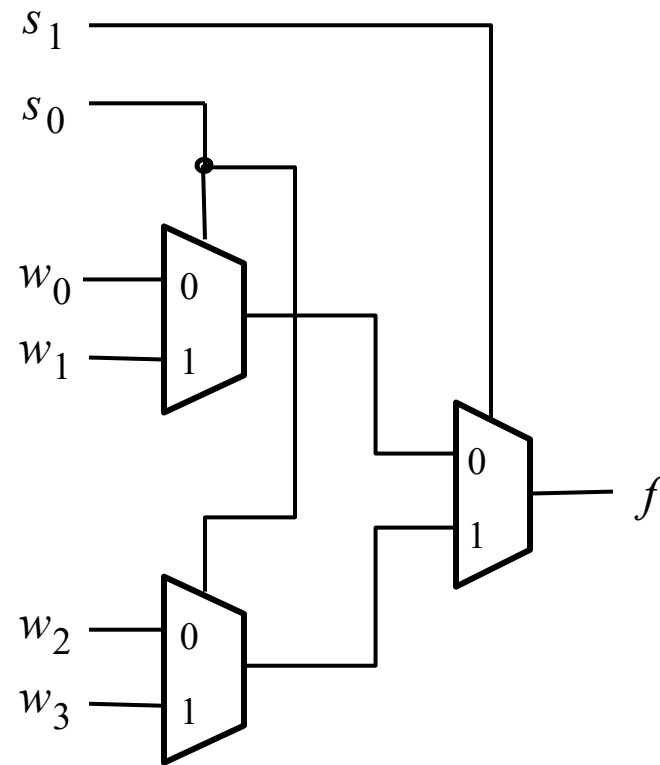
(b) Truth table



(c) Circuit

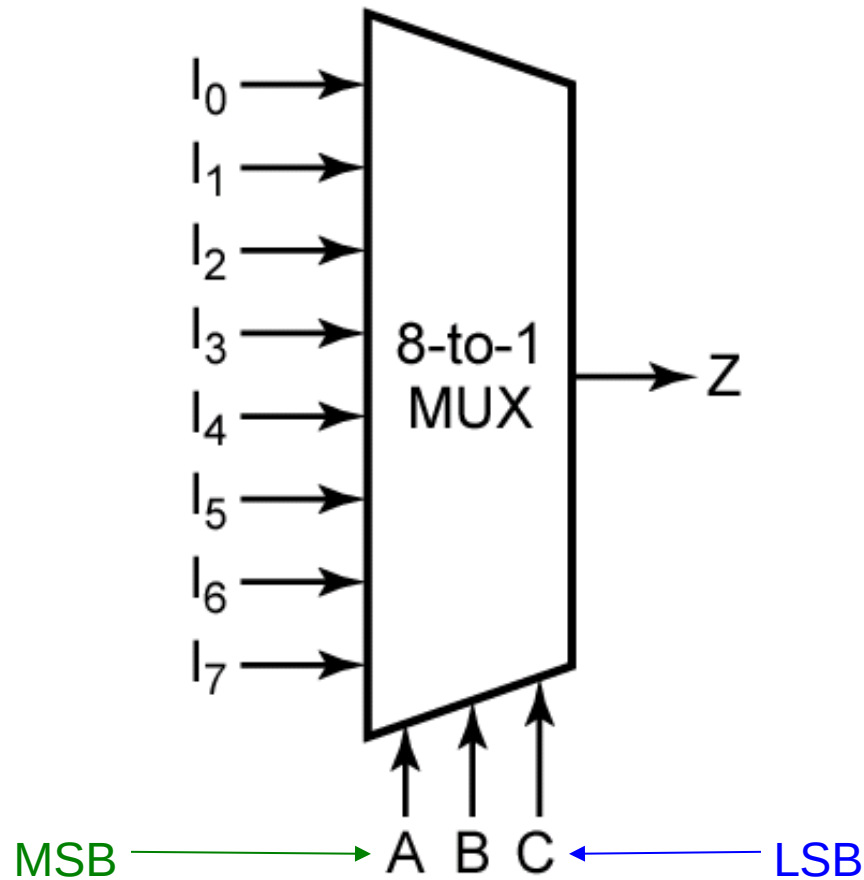
$$f = \overline{s_1} \overline{s_0} w_0 + \overline{s_1} s_0 w_1 + s_1 \overline{s_0} w_2 + s_1 s_0 w_3$$

A 4-to-1 multiplexer.



Using 2-to-1 multiplexers to build a 4-to-1 multiplexer.

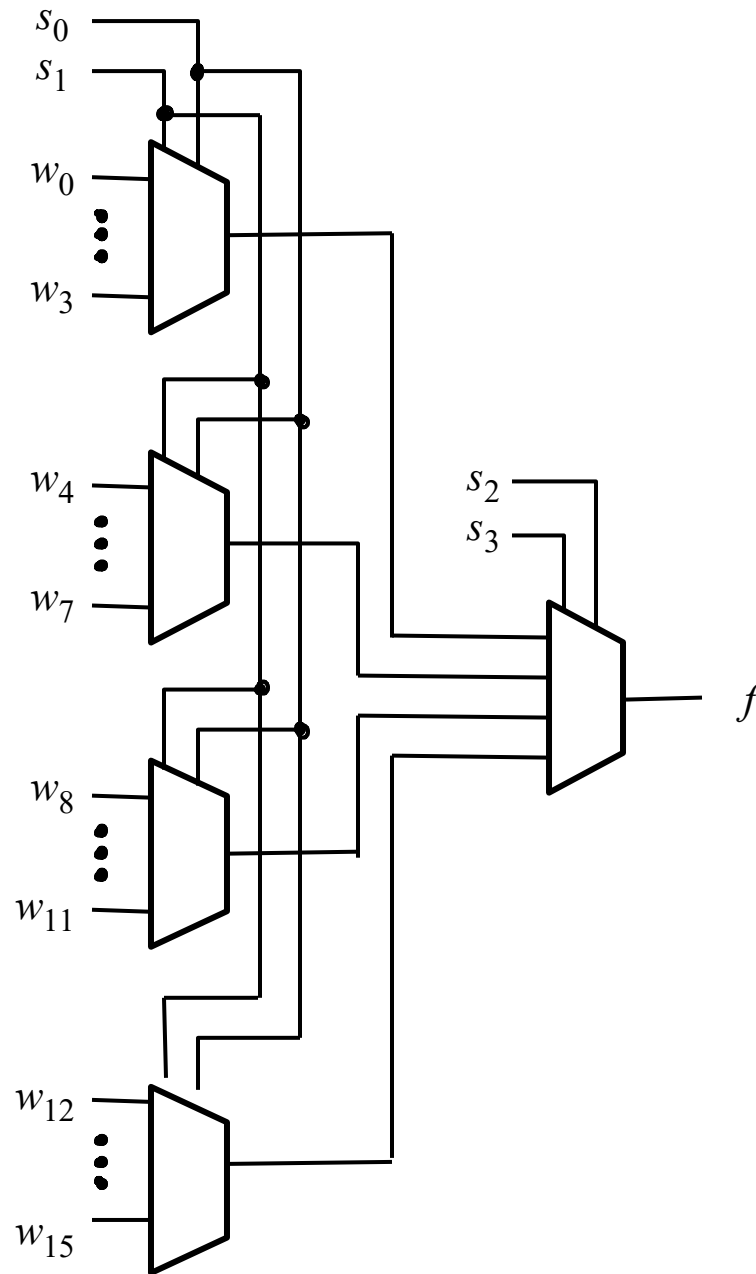
8 to 1 Multiplexers



A	B	C	F
0	0	0	I_0
0	0	1	I_1
0	1	0	I_2
0	1	1	I_3
1	0	0	I_4
1	0	1	I_5
1	1	0	I_6
1	1	1	I_7

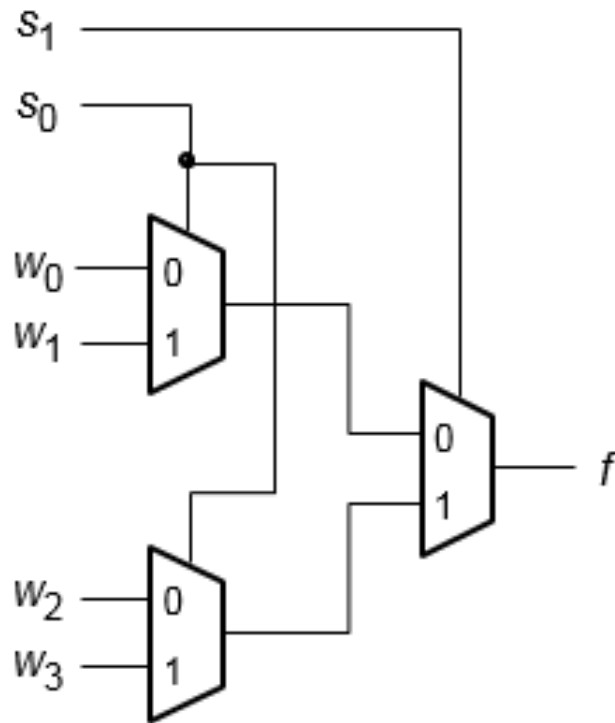
$$Z = A'.B'.C'.I_0 + A'.B'.C.I_1 + A'.B.C'.I_2 + A'.B.C.I_3 + A.B'.C'.I_0 + A.B'.C.I_1 + A'.B.C'.I_2 + A.B.C.I_3$$

Exercise: Design an 8-to-1 multiplexer using
4-to-1 and 2-to-1 multiplexers only.

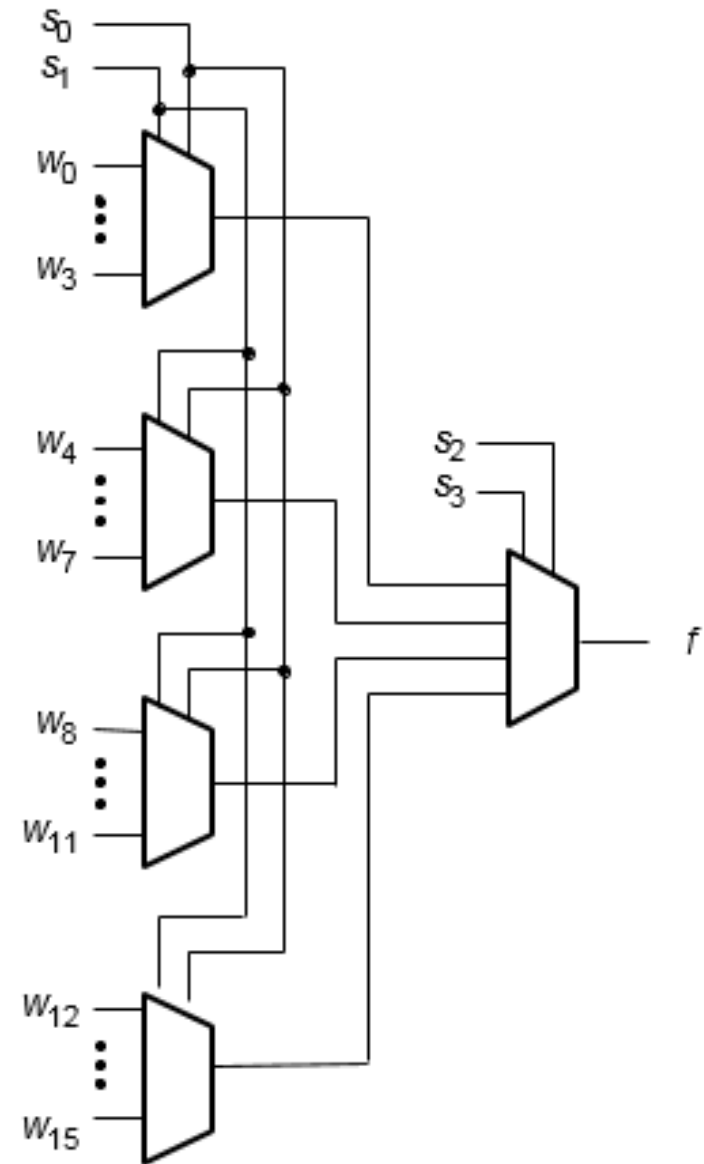


A 16-to-1 multiplexer using 2:1 multiplexer

Bigger MUXes Built from Smaller MUXes

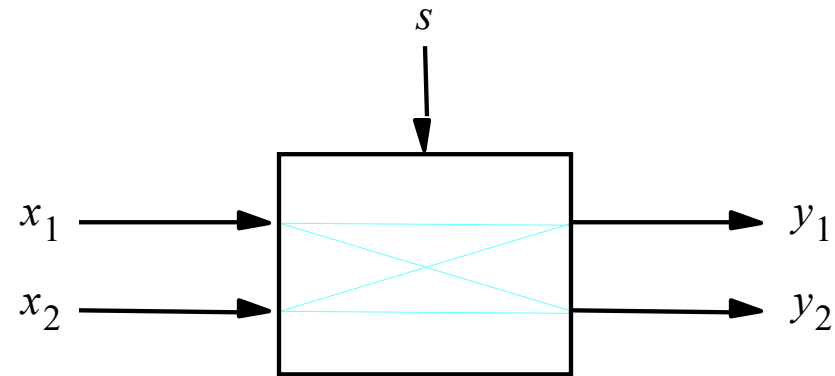


A 4-to-1 MUX built
from 2-to-1 MUXes.

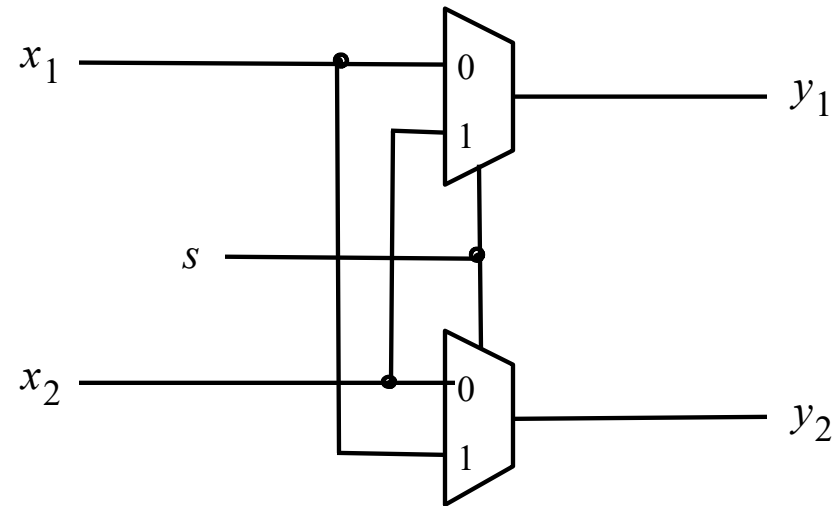


A 16-to-1 MUX built
from 4-to-1 MUXes.

Synthesis of Logic Circuits Using Multiplexers



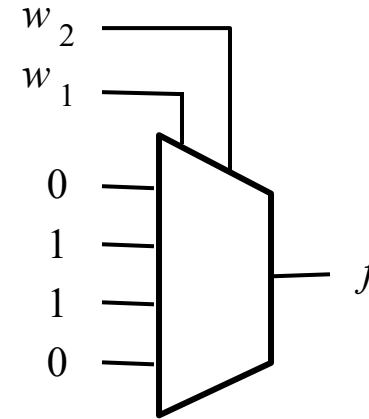
(a) A 2x2 crossbar switch



(b) Implementation using multiplexers

A practical application of multiplexers.

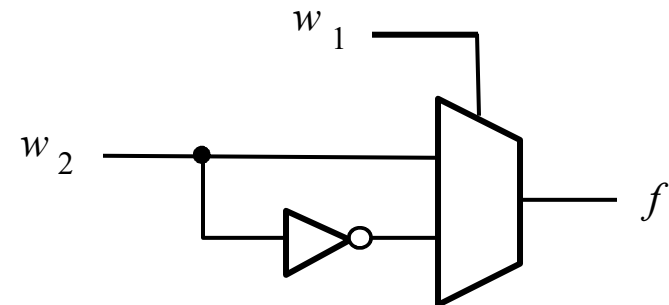
w_1	w_2	f
0	0	0
0	1	1
1	0	1
1	1	0



(a) Implementation using a 4-to-1 multiplexer

w_1	w_2	f		w_1	f
0	0	0	}	0	w_2
0	1	1		1	$\bar{w}_2$
1	0	1	}		
1	1	0			

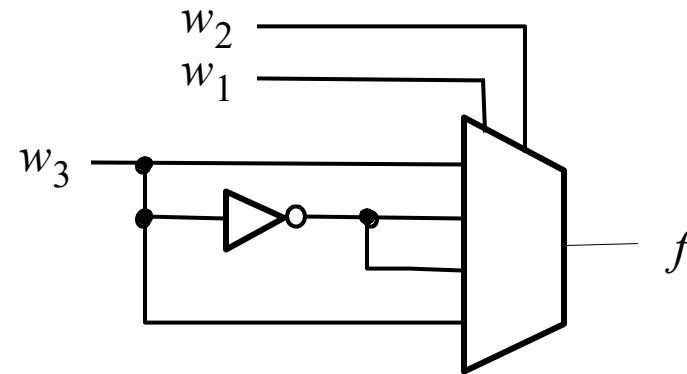
(b) Modified truth table



(c) Circuit using 2:1 mux

w_1	w_2	w_3	f	
0	0	0	0	} w_3
0	0	1	1	
0	1	0	1	} $\bar{w}_3$
0	1	1	0	
1	0	0	1	} $\bar{w}_3$
1	0	1	0	
1	1	0	0	} w_3
1	1	1	1	

(a) Truth table

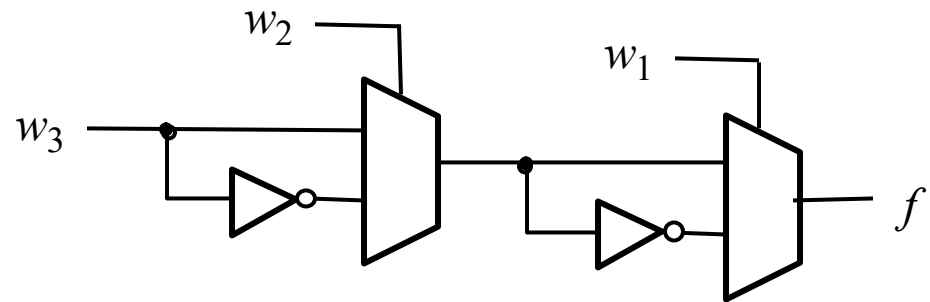


(b) Circuit

Three-input XOR function implemented with a 4-to-1 multiplexer.

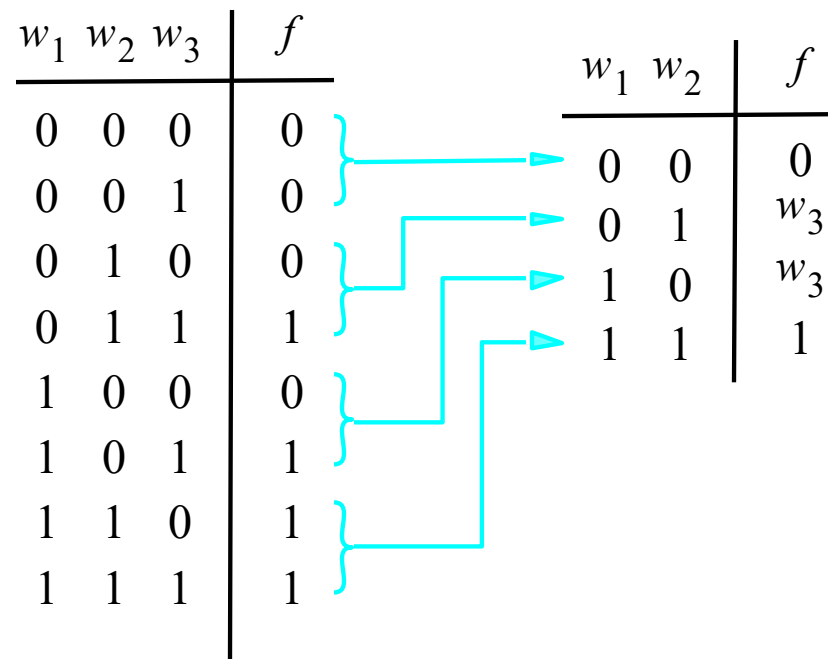
w_1	w_2	w_3	f
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(a) Truth table

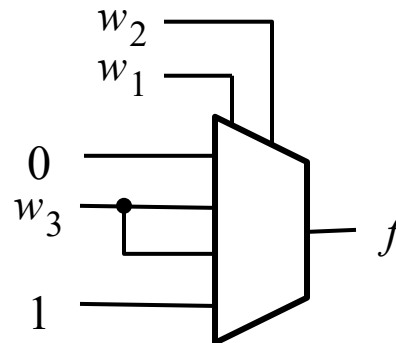


(b) Circuit

Three-input XOR implemented with 2-to-1 multiplexers.



(a) Modified truth table

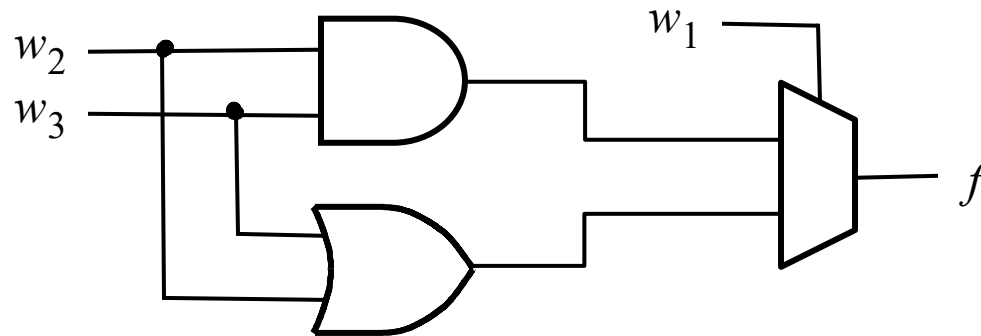


(b) Circuit

Implementation of the three-input majority function using a 4:1 multiplexer.

w_1	w_2	w_3	f		w_1	f
0	0	0	0	{	0	$w_2 w_3$
0	0	1	0			
0	1	0	0			
0	1	1	1			
1	0	0	0	{	1	$w_2 + w_3$
1	0	1	1			
1	1	0	1			
1	1	1	1			

(b) Truth table



(b) Circuit

The three-input majority function implemented using a 2-to-1 multiplexer.

Shannon's Expansion Theorem

Shannon's Expansion Theorem:

Any Boolean function $f(w_1, w_2, \dots, w_n)$ can be written in the form:

$$f(w_1, w_2, \dots, w_n) = w_1' \cdot f(0, w_2, \dots, w_n) + w_1 \cdot f(1, w_2, \dots, w_n)$$

- The term $f(0, w_2, \dots, w_n)$ is known as the cofactor of f with respect to w_1' (also called **negative cofactor** with respect to w_1)
- It is obtained by replacing all occurrences of w_1 in f with 0
- It is denoted as $f_{w_1'}$

- The term $f(1, w_2, \dots, w_n)$ is known as the cofactor of f with respect to w_1 (also called **positive cofactor** with respect to w_1)
- It is obtained by replacing all occurrences of w_1 in f with 1
- It is denoted as f_{w_1}

$$f(w_1, w_2, \dots, w_n) = w_1' \cdot f_{w_1'} + w_1 \cdot f_{w_1}$$

Shannon's Expansion Theorem

Problem: Example 1

Do Shannon's expansion of: $f = w_1 \oplus w_2$ and implement in terms of 2-to-1 MUX.

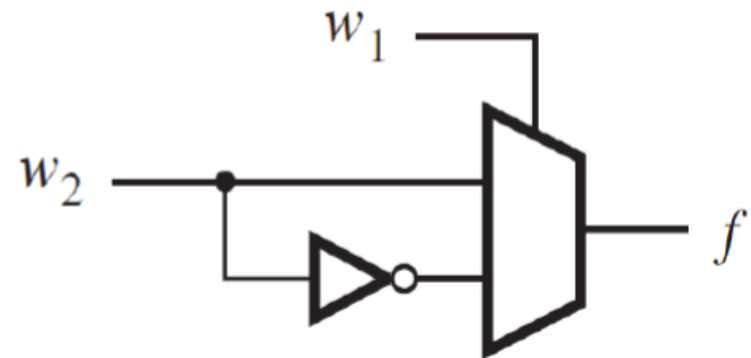
Solution:

$$f = w_1 \oplus w_2 = w_1' \cdot w_2 + w_1 w_2'$$

$$f_{w_1'} = w_2$$

$$f_{w_1} = w_2'$$

$$f = w_1' \cdot f_{w_1'} + w_1 \cdot f_{w_1}$$

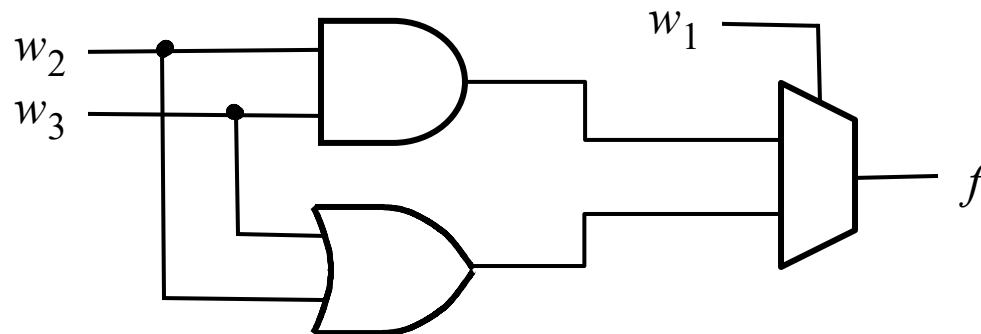


Three-input majority function

w_1	w_2	w_3	f
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

w_1	f
0	$w_2 w_3$
1	$w_2 + w_3$

(b) Truth table

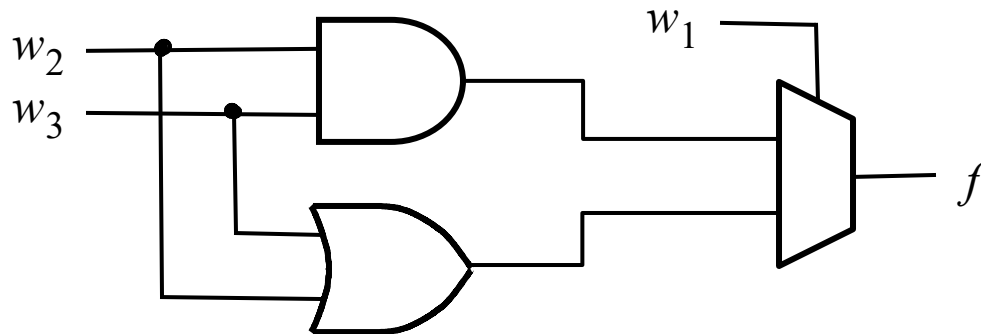


(b) Circuit

Three-input majority function

$$f = \bar{w}_1 w_2 w_3 + w_1 \bar{w}_2 w_3 + w_1 w_2 \bar{w}_3 + w_1 w_2 w_3$$

$$\begin{aligned} f &= \bar{w}_1 (w_2 w_3) + w_1 (\bar{w}_2 w_3 + w_2 \bar{w}_3 + w_2 w_3) \\ &= \bar{w}_1 (w_2 w_3) + w_1 (w_2 + w_3) \end{aligned}$$



Factor and implement the following function with a 2x1 multiplexer

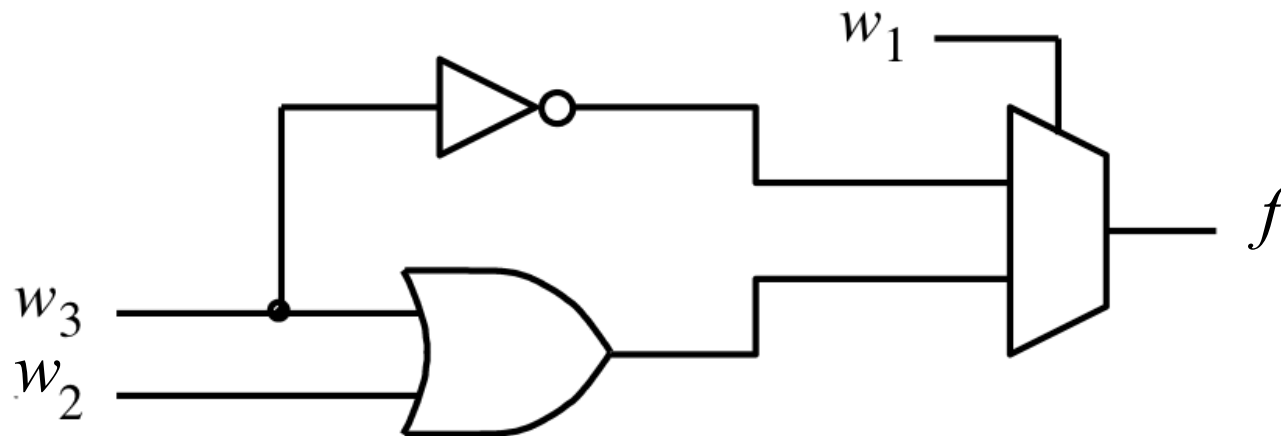
$$f = \overline{w_1}\overline{w_3} + w_1w_2 + w_1w_3$$

Factor and implement the following function with a 2x1 multiplexer

$$f = \overline{w}_1 \overline{w}_3 + w_1 w_2 + w_1 w_3$$

$$\begin{aligned} f &= \overline{w}_1 f_{\overline{w}_1} + w_1 f_{w_1} \\ &= \overline{w}_1 (\overline{w}_3) + w_1 (w_2 + w_3) \end{aligned}$$

Factor and implement the following function
with a 2x1 multiplexer



$$\begin{aligned} f &= \overline{w_1}f_{\overline{w_1}} + w_1f_{w_1} \\ &= \overline{w_1}(\overline{w_3}) + w_1(w_2 + w_3) \end{aligned}$$

Factor and implement the following function with a 4x1 multiplexer

Example 3

$$f = \overline{w}_1 \overline{w}_3 + w_1 w_2 + w_1 w_3$$

Shannon's Expansion Theorem

(In terms of more than one variable)

$$f(w_1, \dots, w_n) = \overline{w}_1 \overline{w}_2 \cdot f(0, 0, w_3, \dots, w_n) + \overline{w}_1 w_2 \cdot f(0, 1, w_3, \dots, w_n) \\ + w_1 \overline{w}_2 \cdot f(1, 0, w_3, \dots, w_n) + w_1 w_2 \cdot f(1, 1, w_3, \dots, w_n)$$

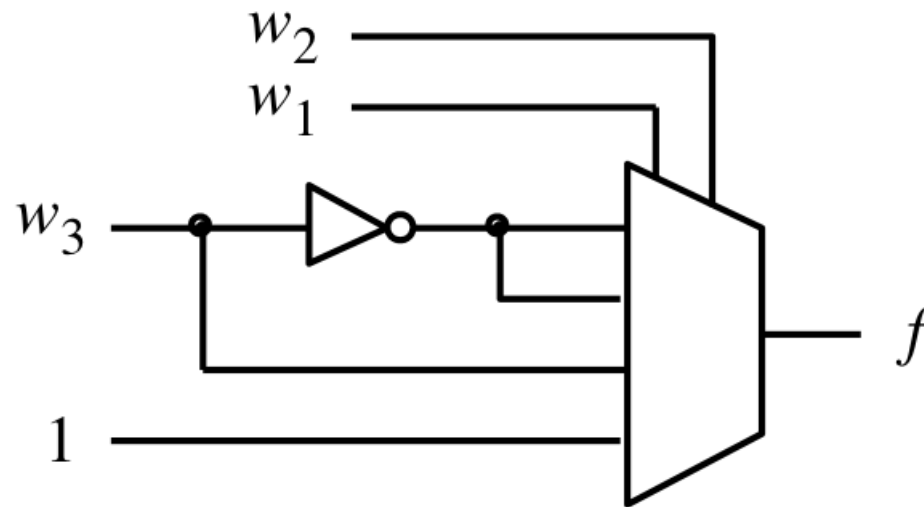
This form is suitable for implementation with a 4x1 multiplexer.

Factor and implement the following function with a 4x1 multiplexer

$$f = \overline{w}_1 \overline{w}_3 + w_1 w_2 + w_1 w_3$$

$$\begin{aligned} f &= \overline{w}_1 \overline{w}_2 f_{\overline{w}_1 \overline{w}_2} + \overline{w}_1 w_2 f_{\overline{w}_1 w_2} + w_1 \overline{w}_2 f_{w_1 \overline{w}_2} + w_1 w_2 f_{w_1 w_2} \\ &= \overline{w}_1 \overline{w}_2 (\overline{w}_3) + \overline{w}_1 w_2 (\overline{w}_3) + w_1 \overline{w}_2 (w_3) + w_1 w_2 (1) \end{aligned}$$

Factor and implement the following function with a 4x1 multiplexer



$$\begin{aligned}
 f &= \overline{w_1}\overline{w_2}f_{\overline{w_1}\overline{w_2}} + \overline{w_1}w_2f_{\overline{w_1}w_2} + w_1\overline{w_2}f_{w_1\overline{w_2}} + w_1w_2f_{w_1w_2} \\
 &= \overline{w_1}\overline{w_2}(\overline{w_3}) + \overline{w_1}w_2(\overline{w_3}) + w_1\overline{w_2}(w_3) + w_1w_2(1)
 \end{aligned}$$

Factor and implement the following function using only 2x1 multiplexers

Example 4

$$f = w_1 w_2 + w_1 w_3 + w_2 w_3$$

Factor and implement the following function using only 2x1 multiplexers

$$f = w_1 w_2 + w_1 w_3 + w_2 w_3$$

$$\begin{aligned} f &= \overline{w}_1 (w_2 w_3) + w_1 (w_2 + w_3 + w_2 w_3) \\ &= \overline{w}_1 (w_2 w_3) + w_1 (w_2 + w_3) \end{aligned}$$

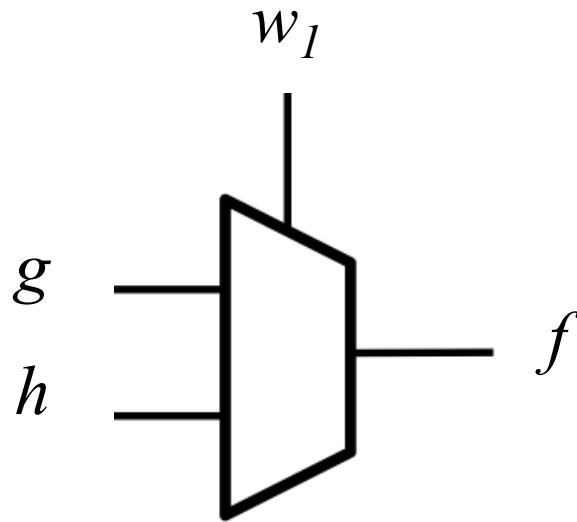
Factor and implement the following function using only 2x1 multiplexers

$$f = w_1 w_2 + w_1 w_3 + w_2 w_3$$

$$\begin{aligned} f &= \bar{w}_1 (w_2 w_3) + w_1 (w_2 + w_3 + w_2 w_3) \\ &= \bar{w}_1 \underbrace{(w_2 w_3)}_g + w_1 \underbrace{(w_2 + w_3)}_h \end{aligned}$$

$$g = w_2 w_3 \quad h = w_2 + w_3$$

Factor and implement the following function using only 2x1 multiplexers



$$\begin{aligned} f &= \bar{w}_1(w_2w_3) + w_1(w_2 + w_3 + w_2w_3) \\ &= \bar{w}_1(\underbrace{w_2w_3}) + w_1(\underbrace{w_2 + w_3}) \end{aligned}$$

$$g = w_2w_3 \quad h = w_2 + w_3$$

Factor and implement the following function
using only 2x1 multiplexers

$$g = w_2 w_3$$

$$h = w_2 + w_3$$

Factor and implement the following function using only 2x1 multiplexers

$$g = w_2 w_3$$



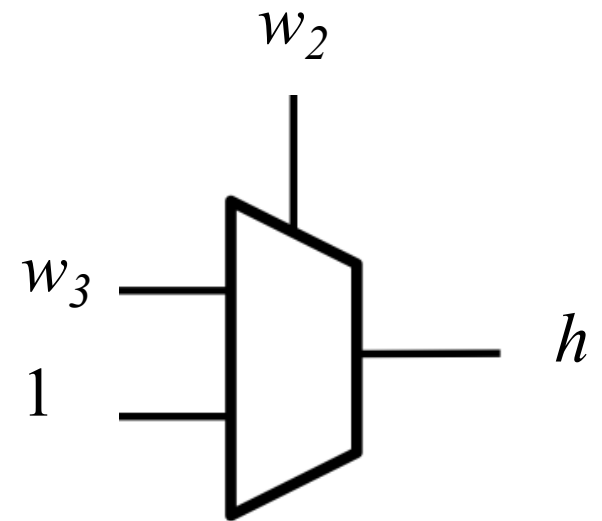
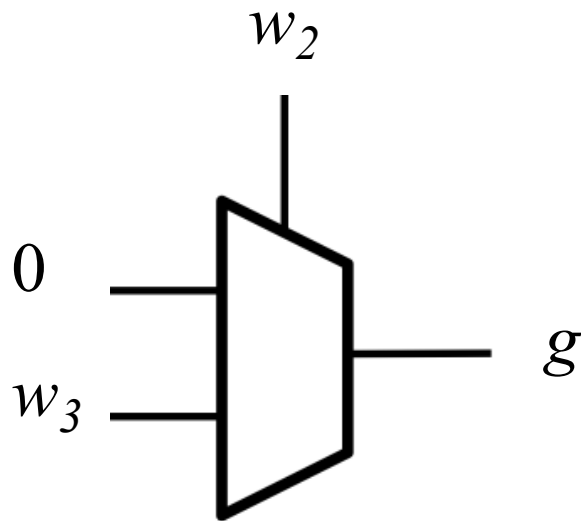
$$g = \bar{w}_2(0) + w_2(w_3)$$

$$h = w_2 + w_3$$



$$h = \bar{w}_2(w_3) + w_2(1)$$

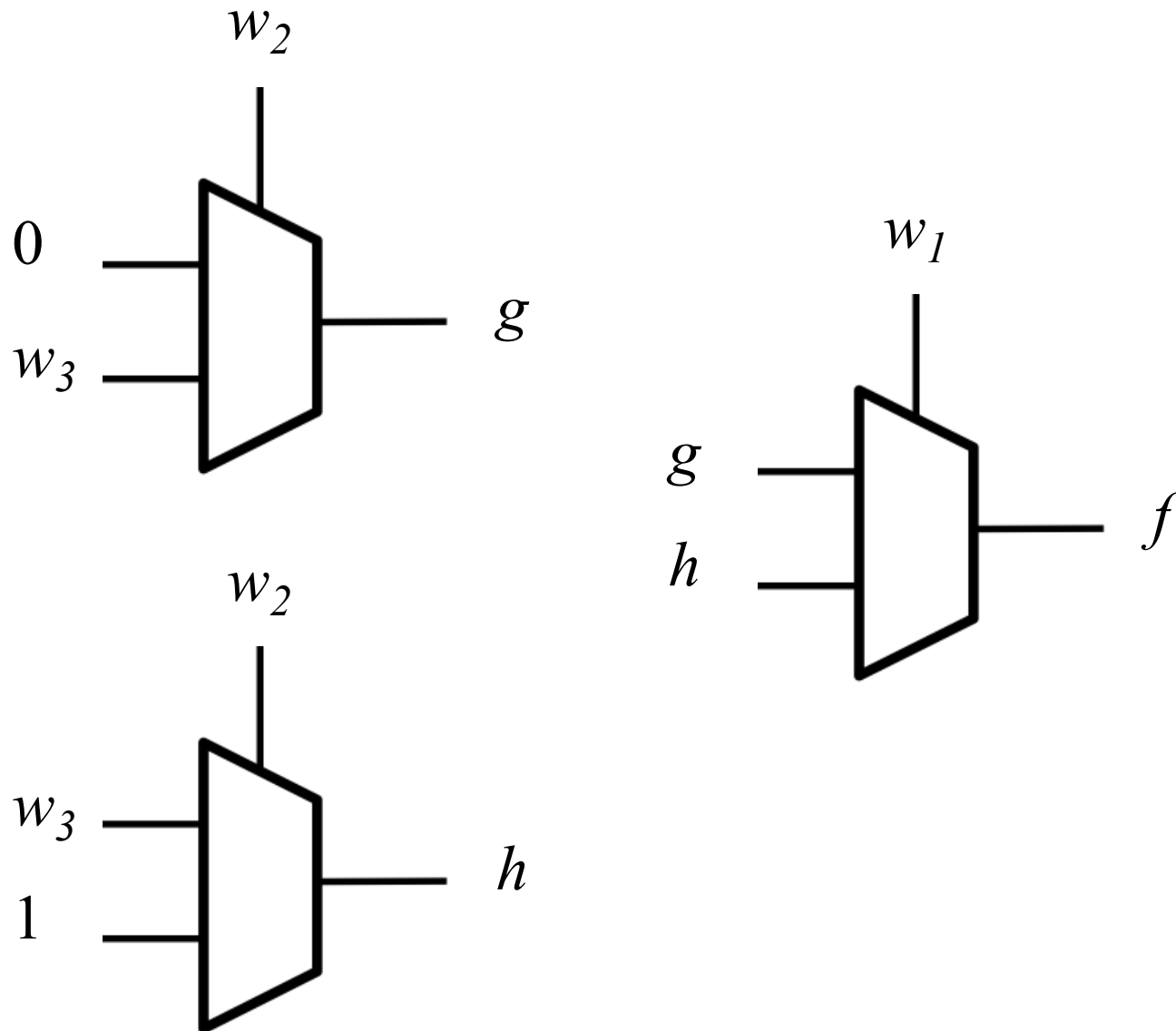
Factor and implement the following function using only 2x1 multiplexers



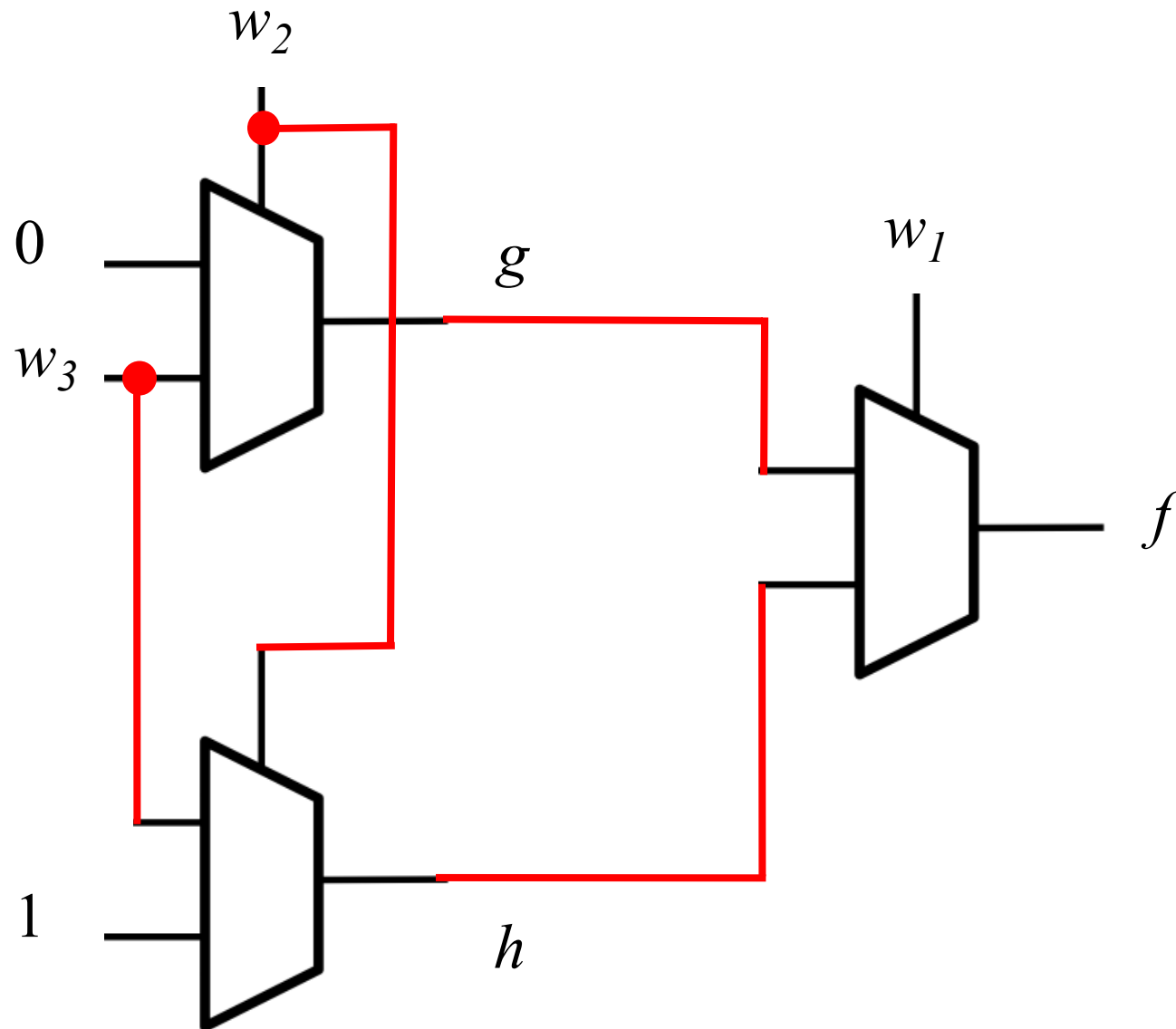
$$g = \overline{w_2}(0) + w_2(w_3)$$

$$h = \overline{w_2}(w_3) + w_2(1)$$

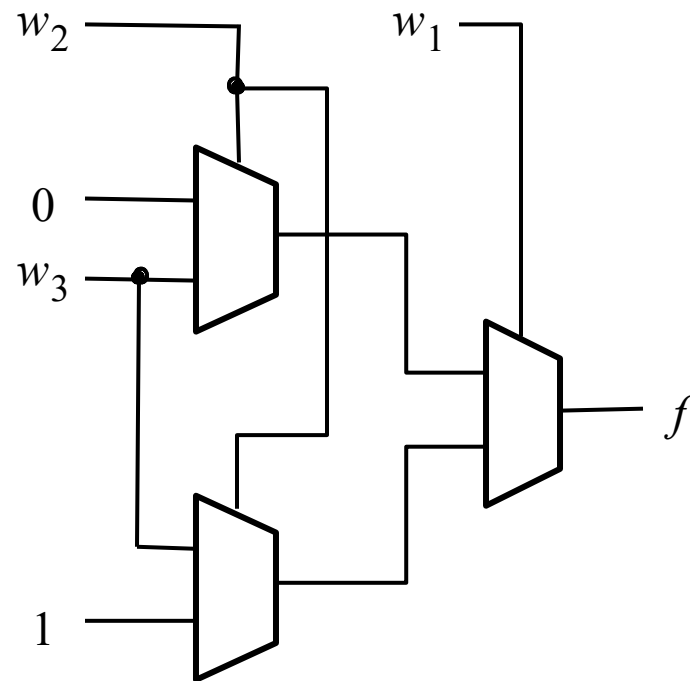
Finally, we are ready to draw the circuit



Finally, we are ready to draw the circuit



Finally, we are ready to draw the circuit



Shannon's Expansion Theorem: Significance of Variables

Problem: Compute the cost of the implementation of the function $f = w_1'w_3 + w_2w_3'$ using only 2-to-1 MUX, when the function is expanded with respect to i) w_1 ii) w_2 iii) w_3

Solution:

ii) Expand with respect to w_2

Similar implementation. Cost is four 2-to-1 MUXes.

Do it yourself...

Shannon's Expansion Theorem: Significance of Variables

Problem: Compute the cost of the implementation of the function $f = w_1'w_3 + w_2w_3'$ using only 2-to-1 MUX, when the function is expanded with respect to i) w_1 ii) w_2 iii) w_3

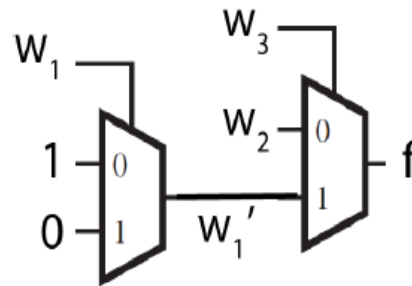
Solution:

iii) Expand with respect to w_3

$$f = w_3' \cdot f_{w_3'} + w_3 \cdot f_{w_3}$$

$$f_{w_3'} = w_2$$

$$f_{w_3} = w_1'$$



Two 2-to-1 MUX are sufficient

Conclusions:

- The order of variables chosen in the recursive Shannon's Expansion has a big impact on the cost of implementation
- In practice, tools try out various orders and come up with a minimized implementation

Implementing Boolean Functions using Multiplexers

Problem 1: Implement the function $f(x, y, z) = \Sigma m(3, 5, 6, 7)$ using minimum 2-to-1 MUX (do not use any other gates)

Solution:

i) Draw the K-map

		yz			
		00	01	11	10
x	0	0	0	1	0
	1	0	1	1	1

ii) Take x as the select input of the MUX:

$$f(x, y, z) = x'(yz) + x(y + z)$$

Can be implemented using three 2-to-1 MUX

iii) Take y as the select input of the MUX:

$$f(x, y, z) = y'(xz) + y(x + z)$$

Can be implemented using three 2-to-1 MUX

iv) Take z as the select input of the MUX:

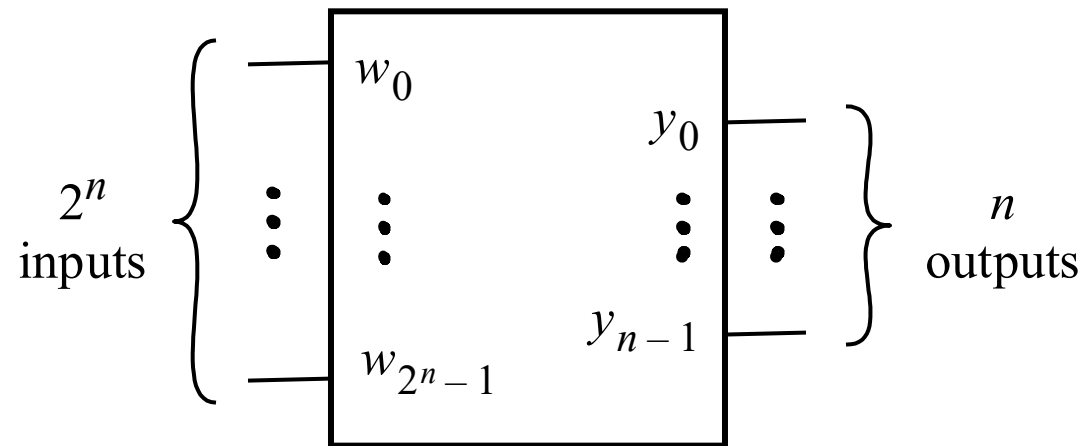
$$f(x, y, z) = z'(xy) + z(x + y)$$

Can be implemented using three 2-to-1 MUX

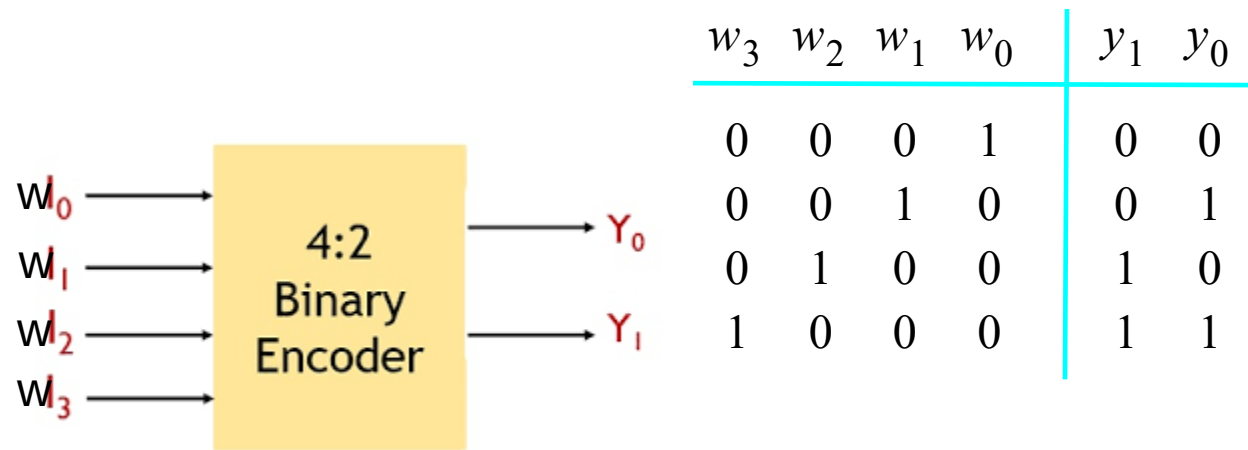
Encoders

Encoders

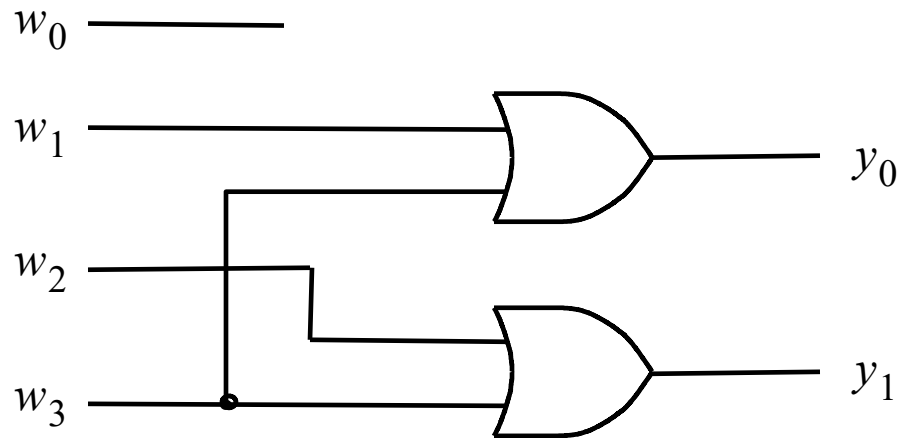
- An **encoder** performs the opposite function of a decoder
- A **binary encoder** encodes information (data) from 2^n inputs into an n -bit code (output)
 - Exactly one of the inputs should have a value of one
 - The outputs represent the binary number that identifies which input is equal to 1
- Encoders reduce the number of bits needed to represent given information
- Practical use: transmitting information in a digital system



A 2^n -to- n binary encoder.



(a) Truth table



(b) Circuit

A 4-to-2 binary encoder.

Encoder Example

- Example: 8-to-3 binary encoder (octal-to-binary)

Inputs								Outputs		
D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0	A_2	A_1	A_0
0	0	0	0	0	0	0	1	0	0	0
0	0	0	0	0	0	1	0	0	0	1
0	0	0	0	0	1	0	0	0	1	0
0	0	0	0	1	0	0	0	0	1	1
0	0	0	1	0	0	0	0	1	0	0
0	0	1	0	0	0	0	0	1	0	1
0	1	0	0	0	0	0	0	1	1	0
1	0	0	0	0	0	0	0	1	1	1

$$\mathbf{A_0 = D_1 + D_3 + D_5 + D_7}$$

$$\mathbf{A_1 = D_2 + D_3 + D_6 + D_7}$$

$$\mathbf{A_2 = D_4 + D_5 + D_6 + D_7}$$

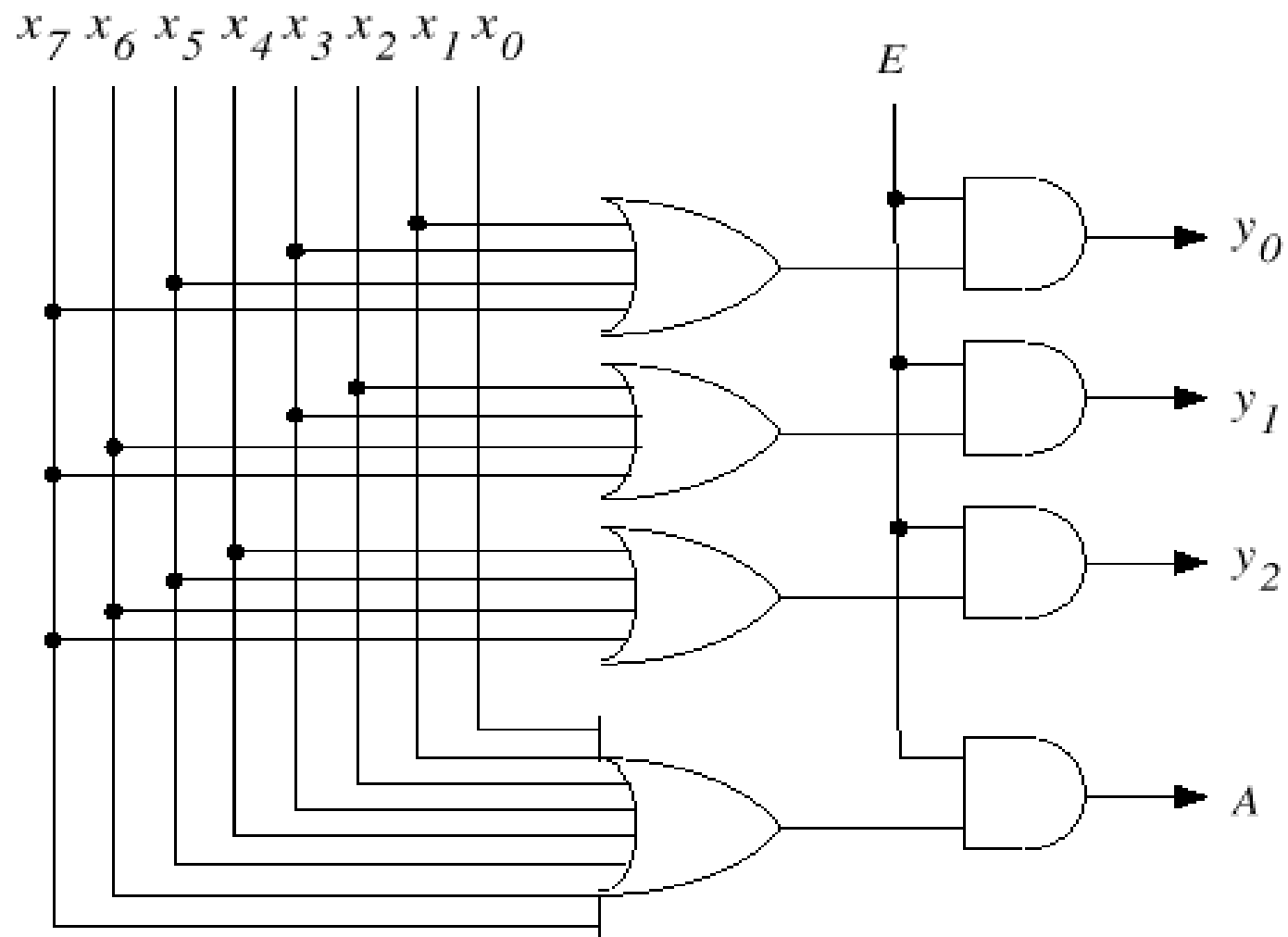
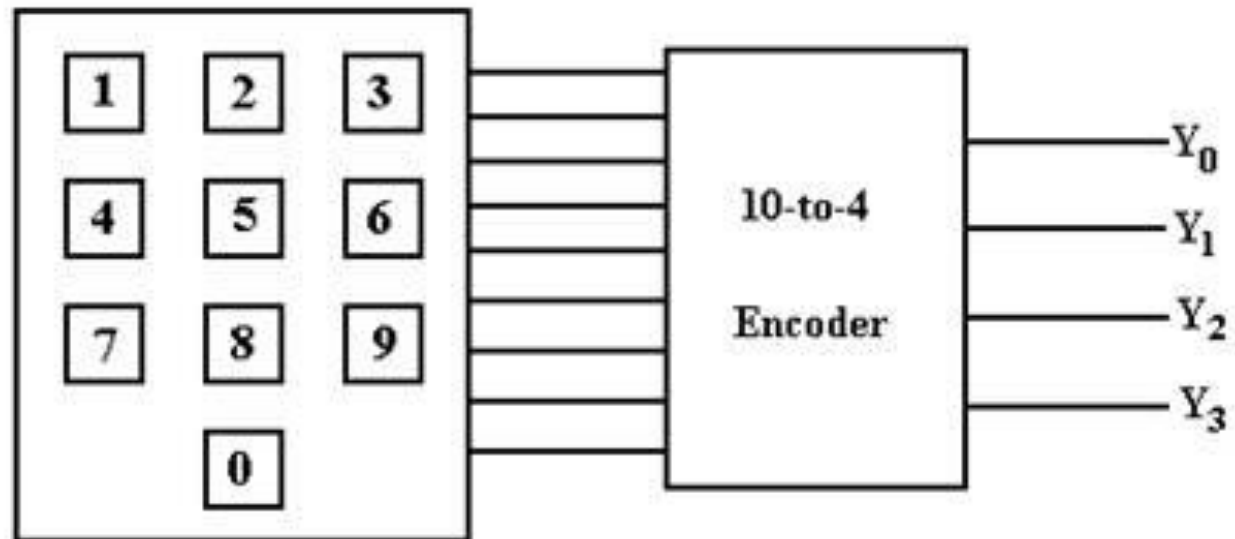


Figure 9.13: Implementation of an 8-input binary encoder.



Priority encoders

- Another useful class of encoders is based on the ***priority*** of the input signals
- In a priority encoder, each input has a priority level associated with it
- The encoder outputs indicate the active input that has the highest priority
 - When an input with a high priority is asserted, the other lower priority inputs are ignored

Priority Encoder

- Assume that w_0 has the lowest priority and w_3 has the highest
- The output z indicates when none of the inputs are 1
- Letting
 - $i_0 = w_3'w_2'w_1'w_0$
 - $i_1 = w_3'w_2'w_1$
 - $i_2 = w_3'w_2$
 - $i_3 = w_3$

w_3	w_2	w_1	w_0	y_1	y_0	z
0	0	0	0	d	d	0
0	0	0	1	0	0	1
0	0	1	x	0	1	1
0	1	x	x	1	0	1
1	x	x	x	1	1	1

$$y_0 = i_1 + i_3, \quad y_1 = i_2 + i_3$$

$$z = i_1 + i_2 + i_3 + i_4$$

Truth table for a 4-to-2 priority encoder.

8-to-3 Priority Encoder

Table 4.6 A priority encoder.

A_0	A_1	A_2	A_3	A_4	A_5	A_6	A_7	Z_0	Z_1	Z_2	NR
0	0	0	0	0	0	0	0	X	X	X	1
X	X	X	X	X	X	X	1	1	1	1	0
X	X	X	X	X	X	1	0	1	1	0	0
X	X	X	X	X	1	0	0	1	0	1	0
X	X	X	X	1	0	0	0	1	0	0	0
X	X	X	1	0	0	0	0	0	1	1	0
X	X	1	0	0	0	0	0	0	1	0	0
X	1	0	0	0	0	0	0	0	0	1	0
1	0	0	0	0	0	0	0	0	0	0	0